Lab 4: Character-level Language Model using RNN

Train a `SimpleRNN` on a small text file (<= 50 KB) to predict the 41st character from 40 preceding characters, then generate new text in the style of the training data.

1. Load text and build char-to-index / index-to-char mappings

In [1]:
```
import os, numpy as np
from tensorflow import keras

np.random.seed(42)
keras.utils.set_random_seed(42)

SEQ_LEN, STEP = 40, 1
PATH = "sample.txt"
assert os.path.getsize(PATH) <= 50 * 1024, "input must be <= 50 KB"

text = open(PATH, encoding="utf-8").read()
chars = sorted(set(text))
char_to_idx = {c: i for i, c in enumerate(chars)}
idx_to_char = {i: c for c, i in char_to_idx.items()}
print(f"Corpus: {len(text)} chars, vocabulary: {len(chars)} unique chars")
print(chars)
```

```
WARNING: All log messages before absl::InitializeLog() is called are written
to STDERR
I0000 00:00:1793590470.569960     602 port.cc:153] oneDNN custom operations
are on. You may see slightly different numerical results due to floating-poi
nt round-off errors from different computation orders. To turn them off, set
the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1793590484.736724     602 cpu_feature_guard.cc:227] This TensorF
low binary is optimized to use available CPU instructions in performance-cri
tical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 A
VX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, reb
uild TensorFlow with the appropriate compiler flags.
WARNING: All log messages before absl::InitializeLog() is called are written
to STDERR
I0000 00:00:1793590493.053352     602 port.cc:153] oneDNN custom operations
are on. You may see slightly different numerical results due to floating-poi
nt round-off errors from different computation orders. To turn them off, set
the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

```
Corpus: 2444 chars, vocabulary: 49 unique chars
['\n', ' ', '!', "'", ',', '-', '.', ':', ';', '?', 'A', 'B', 'D', 'F', 'H',
'I', 'L', 'M', 'N', 'O', 'R', 'S', 'T', 'W', 'Y', 'a', 'b', 'c', 'd', 'e',
'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'r', 's', 't', 'u',
'v', 'w', 'x', 'y']
```

2-3. Create training sequences (40 chars → 41st) and vectorize

In [2]:
```
inputs, targets = [], []
for i in range(0, len(text) - SEQ_LEN, STEP):
    inputs.append(text[i:i + SEQ_LEN])
    targets.append(text[i + SEQ_LEN])

vocab = len(chars)
x = np.zeros((len(inputs), SEQ_LEN, vocab), dtype=np.float32)
y = np.zeros((len(inputs), vocab), dtype=np.float32)
for n, (window, nxt) in enumerate(zip(inputs, targets)):
    for t, ch in enumerate(window):
        x[n, t, char_to_idx[ch]] = 1.0
    y[n, char_to_idx[nxt]] = 1.0

print("x:", x.shape, " y:", y.shape)
print(repr(inputs[0]), "->", repr(targets[0]))
```

```
x: (2404, 40, 49)  y: (2404, 49)
"Shall I compare thee to a summer's day?\n" -> 'T'
```

4. Build the RNN model

The first `SimpleRNN` returns the whole sequence so the second can consume it.

In [3]:
```
model = keras.Sequential([
    keras.layers.Input(shape=(SEQ_LEN, vocab)),
    keras.layers.SimpleRNN(256, return_sequences=True),
    keras.layers.SimpleRNN(256),
    keras.layers.Dense(vocab, activation="softmax"),
])
model.compile(loss="categorical_crossentropy",
              optimizer=keras.optimizers.Adam(learning_rate=1e-3, clipnorm=1
              metrics=["accuracy"])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Par
simple_rnn (SimpleRNN)	(None, 40, 256)	78
simple_rnn_1 (SimpleRNN)	(None, 256)	131
dense (Dense)	(None, 49)	12

Total params: 222,257 (868.19 KB)

Trainable params: 222,257 (868.19 KB)

Non-trainable params: 0 (0.00 B)

5. Train the model

In [4]:
```
history = model.fit(x, y, batch_size=64, epochs=100, verbose=2)
```

```
Epoch 1/100
38/38 - 4s - 92ms/step - accuracy: 0.1423 - loss: 3.2510
Epoch 2/100
38/38 - 1s - 28ms/step - accuracy: 0.1797 - loss: 3.0637
Epoch 3/100
38/38 - 1s - 27ms/step - accuracy: 0.2151 - loss: 2.8688
Epoch 4/100
38/38 - 1s - 25ms/step - accuracy: 0.2708 - loss: 2.5909
Epoch 5/100
38/38 - 1s - 27ms/step - accuracy: 0.3465 - loss: 2.2864
Epoch 6/100
38/38 - 1s - 26ms/step - accuracy: 0.4205 - loss: 1.9947
Epoch 7/100
38/38 - 1s - 24ms/step - accuracy: 0.4867 - loss: 1.7481
Epoch 8/100
38/38 - 1s - 25ms/step - accuracy: 0.5661 - loss: 1.5169
Epoch 9/100
38/38 - 1s - 27ms/step - accuracy: 0.6302 - loss: 1.2957
Epoch 10/100
38/38 - 1s - 27ms/step - accuracy: 0.7138 - loss: 1.0978
Epoch 11/100
38/38 - 1s - 27ms/step - accuracy: 0.7683 - loss: 0.9162
Epoch 12/100
38/38 - 1s - 26ms/step - accuracy: 0.8170 - loss: 0.7609
Epoch 13/100
38/38 - 1s - 24ms/step - accuracy: 0.8669 - loss: 0.6207
Epoch 14/100
38/38 - 1s - 25ms/step - accuracy: 0.9097 - loss: 0.5112
Epoch 15/100
38/38 - 1s - 26ms/step - accuracy: 0.9280 - loss: 0.4177
Epoch 16/100
38/38 - 1s - 27ms/step - accuracy: 0.9438 - loss: 0.3648
Epoch 17/100
38/38 - 1s - 25ms/step - accuracy: 0.9434 - loss: 0.3409
Epoch 18/100
38/38 - 1s - 28ms/step - accuracy: 0.9671 - loss: 0.2651
Epoch 19/100
38/38 - 1s - 26ms/step - accuracy: 0.9759 - loss: 0.2044
Epoch 20/100
38/38 - 1s - 27ms/step - accuracy: 0.9817 - loss: 0.1783
Epoch 21/100
38/38 - 1s - 26ms/step - accuracy: 0.9780 - loss: 0.1590
Epoch 22/100
38/38 - 1s - 28ms/step - accuracy: 0.9838 - loss: 0.1387
Epoch 23/100
38/38 - 1s - 30ms/step - accuracy: 0.9875 - loss: 0.1315
```

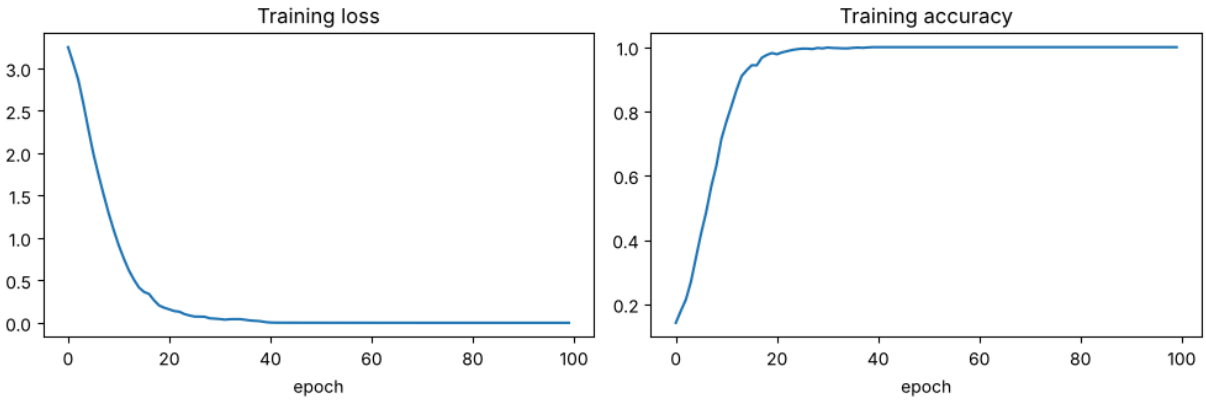
```
Epoch 24/100
38/38 - 1s - 28ms/step - accuracy: 0.9913 - loss: 0.1035
Epoch 25/100
38/38 - 1s - 28ms/step - accuracy: 0.9938 - loss: 0.0870
Epoch 26/100
38/38 - 1s - 29ms/step - accuracy: 0.9954 - loss: 0.0739
Epoch 27/100
38/38 - 1s - 27ms/step - accuracy: 0.9954 - loss: 0.0734
Epoch 28/100
38/38 - 1s - 31ms/step - accuracy: 0.9942 - loss: 0.0716
Epoch 29/100
38/38 - 1s - 27ms/step - accuracy: 0.9975 - loss: 0.0535
Epoch 30/100
38/38 - 1s - 27ms/step - accuracy: 0.9963 - loss: 0.0496
Epoch 31/100
38/38 - 1s - 25ms/step - accuracy: 0.9988 - loss: 0.0449
Epoch 32/100
38/38 - 1s - 27ms/step - accuracy: 0.9975 - loss: 0.0378
Epoch 33/100
38/38 - 1s - 28ms/step - accuracy: 0.9971 - loss: 0.0431
Epoch 34/100
38/38 - 1s - 28ms/step - accuracy: 0.9963 - loss: 0.0437
Epoch 35/100
38/38 - 1s - 27ms/step - accuracy: 0.9963 - loss: 0.0437
Epoch 36/100
38/38 - 1s - 26ms/step - accuracy: 0.9979 - loss: 0.0361
Epoch 37/100
38/38 - 1s - 27ms/step - accuracy: 0.9988 - loss: 0.0277
Epoch 38/100
38/38 - 1s - 27ms/step - accuracy: 0.9979 - loss: 0.0237
Epoch 39/100
38/38 - 1s - 29ms/step - accuracy: 0.9992 - loss: 0.0193
Epoch 40/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 0.0091
Epoch 41/100
38/38 - 1s - 28ms/step - accuracy: 1.0000 - loss: 0.0037
Epoch 42/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 0.0022
Epoch 43/100
38/38 - 1s - 30ms/step - accuracy: 1.0000 - loss: 0.0019
Epoch 44/100
38/38 - 1s - 29ms/step - accuracy: 1.0000 - loss: 0.0017
Epoch 45/100
38/38 - 1s - 30ms/step - accuracy: 1.0000 - loss: 0.0016
Epoch 46/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 0.0015
Epoch 47/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 0.0014
Epoch 48/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 0.0014
Epoch 49/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 0.0013
Epoch 50/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 0.0013
Epoch 51/100
38/38 - 1s - 25ms/step - accuracy: 1.0000 - loss: 0.0012
```

```
Epoch 52/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 0.0012
Epoch 53/100
38/38 - 1s - 25ms/step - accuracy: 1.0000 - loss: 0.0011
Epoch 54/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 0.0011
Epoch 55/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 0.0010
Epoch 56/100
38/38 - 1s - 28ms/step - accuracy: 1.0000 - loss: 0.0010
Epoch 57/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 9.8057e-04
Epoch 58/100
38/38 - 1s - 35ms/step - accuracy: 1.0000 - loss: 9.5070e-04
Epoch 59/100
38/38 - 1s - 34ms/step - accuracy: 1.0000 - loss: 9.2245e-04
Epoch 60/100
38/38 - 1s - 29ms/step - accuracy: 1.0000 - loss: 8.9564e-04
Epoch 61/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 8.7015e-04
Epoch 62/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 8.4586e-04
Epoch 63/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 8.2267e-04
Epoch 64/100
38/38 - 1s - 29ms/step - accuracy: 1.0000 - loss: 8.0048e-04
Epoch 65/100
38/38 - 1s - 29ms/step - accuracy: 1.0000 - loss: 7.7923e-04
Epoch 66/100
38/38 - 1s - 30ms/step - accuracy: 1.0000 - loss: 7.5885e-04
Epoch 67/100
38/38 - 1s - 29ms/step - accuracy: 1.0000 - loss: 7.3926e-04
Epoch 68/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 7.2043e-04
Epoch 69/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 7.0230e-04
Epoch 70/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 6.8482e-04
Epoch 71/100
38/38 - 1s - 25ms/step - accuracy: 1.0000 - loss: 6.6796e-04
Epoch 72/100
38/38 - 1s - 29ms/step - accuracy: 1.0000 - loss: 6.5167e-04
Epoch 73/100
38/38 - 1s - 31ms/step - accuracy: 1.0000 - loss: 6.3594e-04
Epoch 74/100
38/38 - 1s - 29ms/step - accuracy: 1.0000 - loss: 6.2072e-04
Epoch 75/100
38/38 - 1s - 29ms/step - accuracy: 1.0000 - loss: 6.0599e-04
Epoch 76/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 5.9173e-04
Epoch 77/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 5.7790e-04
Epoch 78/100
38/38 - 1s - 25ms/step - accuracy: 1.0000 - loss: 5.6450e-04
Epoch 79/100
38/38 - 1s - 35ms/step - accuracy: 1.0000 - loss: 5.5149e-04
```

```
Epoch 80/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 5.3887e-04
Epoch 81/100
38/38 - 1s - 25ms/step - accuracy: 1.0000 - loss: 5.2661e-04
Epoch 82/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 5.1471e-04
Epoch 83/100
38/38 - 1s - 29ms/step - accuracy: 1.0000 - loss: 5.0313e-04
Epoch 84/100
38/38 - 1s - 31ms/step - accuracy: 1.0000 - loss: 4.9187e-04
Epoch 85/100
38/38 - 1s - 29ms/step - accuracy: 1.0000 - loss: 4.8091e-04
Epoch 86/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 4.7025e-04
Epoch 87/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 4.5988e-04
Epoch 88/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 4.4978e-04
Epoch 89/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 4.3993e-04
Epoch 90/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 4.3035e-04
Epoch 91/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 4.2100e-04
Epoch 92/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 4.1189e-04
Epoch 93/100
38/38 - 1s - 24ms/step - accuracy: 1.0000 - loss: 4.0301e-04
Epoch 94/100
38/38 - 1s - 24ms/step - accuracy: 1.0000 - loss: 3.9434e-04
Epoch 95/100
38/38 - 1s - 35ms/step - accuracy: 1.0000 - loss: 3.8588e-04
Epoch 96/100
38/38 - 1s - 25ms/step - accuracy: 1.0000 - loss: 3.7763e-04
Epoch 97/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 3.6958e-04
Epoch 98/100
38/38 - 1s - 26ms/step - accuracy: 1.0000 - loss: 3.6172e-04
Epoch 99/100
38/38 - 1s - 36ms/step - accuracy: 1.0000 - loss: 3.5404e-04
Epoch 100/100
38/38 - 1s - 27ms/step - accuracy: 1.0000 - loss: 3.4655e-04
```

In [5]:

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(1, 2, figsize=(10, 3.5))
ax[0].plot(history.history["loss"]); ax[0].set_title("Training loss"); ax[0]
ax[1].plot(history.history["accuracy"]); ax[1].set_title("Training accuracy"
plt.tight_layout(); plt.show()
```



6. Text generation function

Temperature sampling: low = conservative, high = more random.

In [6]:
```
def sample(probs, temperature=1.0):
    p = np.log(np.asarray(probs, dtype=np.float64) + 1e-9) / temperature
    p = np.exp(p - p.max()); p /= p.sum()
    return int(np.random.choice(len(p), p=p))

def generate(seed, length=400, temperature=0.7):
    window = "".join(c for c in seed if c in char_to_idx)[-SEQ_LEN:]
    out = window
    for _ in range(length):
        xi = np.zeros((1, SEQ_LEN, vocab), dtype=np.float32)
        pad = SEQ_LEN - len(window)
        for t, ch in enumerate(window):
            xi[0, pad + t, char_to_idx[ch]] = 1.0
        nxt = idx_to_char[sample(model.predict(xi, verbose=0)[0], temperatur
        out += nxt
        window = (window + nxt)[-SEQ_LEN:]
    return out
```

7. Generate new text and validate

In [7]:
```
seed = text[:SEQ_LEN]
for temp in (0.3, 0.7, 1.0):
    print(f"--- temperature {temp} ---")
    print(generate(seed, temperature=temp))
    print()
```

```
--- temperature 0.3 ---
```

```
Shall I compare thee to a summer's day?
Thou art more lovely and more temperate:
Rough winds do shake the darling buds of May,
And summer's lease hath all too short a date;
Sometime too hot the eye of heaven shines,
And often is his gold complexion dimm'd;
And every fair from fair sometime declines,
By chance or nature's changing course untrimm'd;
But thy eternal summer shall not fade,
Nor lose possession of that fair thou ow'st;
Nor sh

--- temperature 0.7 ---
Shall I compare thee to a summer's day?
Thou art more lovely and more temperate:
Rough winds do shake the darling buds of May,
And summer's lease hath all too short a date;
Sometime too hot the eye of heaven shines,
And often is his gold complexion dimm'd;
And every fair from fair sometime declines,
By chance or nature's changing course untrimm'd;
But thy eternal summer shall not fade,
Nor lose possession of that fair thou ow'st;
Nor sh

--- temperature 1.0 ---
Shall I compare thee to a summer's day?
Thou art more lovely and more temperate:
Rough winds do shake the darling buds of May,
And summer's lease hath all too short a date;
Sometime too hot the eye of heaven shines,
And often is his gold complexion dimm'd;
And every fair from fair sometime declines,
By chance or nature's changing course untrimm'd;
But thy eternal summer shall not fade,
Nor lose possession of that fair thou ow'st;
Nor sh
```

In [8]:

```python
gen = generate(seed, length=500, temperature=0.5)[SEQ_LEN:]
train_words = set(text.lower().split())
words = gen.lower().split()
hit = sum(w in train_words for w in words) / max(len(words), 1)
print(f"{hit:.0%} of generated words appear in the training text")
```

```
99% of generated words appear in the training text
```

Observation. The model reproduces the vocabulary, line breaks, punctuation and rhythm of the sonnets. Lower temperatures give near-verbatim lines (the corpus is only ~2.4 KB, so the network largely memorises it); higher temperatures give more varied but noisier text. A larger corpus (up to 50 KB) would produce less memorised, more novel output.